

Práctica 2 — Programación Distribuida y Tiempo Real

Sockets: comunicaciones en red, Vagrant y experimentación de tiempos (Versión Java)

Asistente de IA utilizado: Claude (Anthropic) — Antigravity IDE.
Se deja constancia de su uso en los puntos 1 y 4 donde el enunciado lo requiere explícitamente.

Ejercicio 1 — Ubuntu LTS más reciente en Vagrant

Versión seleccionada

Aspecto	Valor
Box de Vagrant	ubuntu/noble64
Versión de Ubuntu	24.04 LTS “Noble Numbat”
Origen	Canonical (publicado en Vagrant Cloud)
Kernel	Linux 6.8+
Soporte LTS	Hasta abril de 2029 (5 años)

Comparación con la versión usada en el TP1

En el TP1 se usó `ubuntu/jammy64` (Ubuntu 22.04 LTS “Jammy Jellyfish”). Las principales diferencias son:

Aspecto	Ubuntu 22.04 (Jammy)	Ubuntu 24.04 (Noble)
Kernel	5.15	6.8

GCC	11.x	13.x / 14.x
Java (OpenJDK)	11 / 17	17 / 21
Python	3.10	3.12
Systemd	249	255+
OpenSSL	3.0	3.2+
Soporte hasta	Abril 2027	Abril 2029

Consulta a la IA: ¿qué versión conviene más?

Pregunta realizada a Claude (Anthropic):

“¿Para experimentación con sockets TCP en Java dentro de Vagrant, es mejor usar Ubuntu 22.04 LTS o Ubuntu 24.04 LTS? ¿Por qué?”

Respuesta resumida de la IA:

Para experimentación académica con sockets TCP en Java, **ambas versiones son perfectamente válidas**. Sin embargo, Ubuntu 24.04 LTS tiene algunas ventajas marginales:

- **Kernel 6.8** incluye mejoras en el stack de red TCP (TCP-AO, mejoras en BBRv3) que pueden afectar sutilmente las mediciones de latencia en escenarios de alta carga.
- **OpenJDK 21** (LTS) viene disponible por defecto, con mejoras en el garbage collector (ZGC generacional) y mejor rendimiento en operaciones de I/O respecto a versiones anteriores.
- **Soporte más largo** (2 años adicionales de actualizaciones de seguridad).

La IA también señaló que Ubuntu 22.04 puede ser preferible si se busca **máxima estabilidad** y compatibilidad con software legacy, ya que tiene más tiempo en producción y bugs resueltos. Para nuestro caso de uso (experimentos simples de sockets en una red local/virtual), la diferencia práctica es mínima.

Comentario propio: La respuesta de la IA es razonable. Para esta práctica, la elección de 24.04 se justifica simplemente porque el enunciado pide usar una **versión más reciente** que la del TP1.

Vagrantfile utilizado

```
# Vagrantfile.punto1
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/noble64"
  config.vm.hostname = "noble-tp2"
  config.vm.provider "virtualbox" do |vb|
    vb.memory = "512"
    vb.name = "TP2-Ubuntu-Noble-24.04"
  end
  config.vm.provision "shell", inline: <<-SHELL
    apt-get update
    apt-get install -y net-tools joe zip build-essential default-jdk
  SHELL
end
```

Nota: Se agrega `default-jdk` al provisioning para tener `javac` y `java` disponibles dentro de la VM.

Cómo ejecutarlo

```
# Copiar el Vagrantfile
cp Vagrantfile.punto1 Vagrantfile

# Levantar la VM (descarga la imagen la primera vez)
vagrant up

# Conectarse por SSH
vagrant ssh

# Dentro de la VM, verificar la versión:
lsb_release -a
uname -r
java -version
```

Capturas de pantalla

TODO: Insertar aquí dos capturas:

1. La máquina virtual "TP2-Ubuntu-Noble-24.04" visible en la GUI de VirtualBox.

2. La terminal SSH mostrando la salida de `lsb_release -a` , `uname -r` y `java -version` .

Ejercicio 2 — Experimentos entre dos computadoras físicas

Nota: Este ejercicio requiere dos computadoras físicas conectadas en red. Se utilizan los programas `ServerTimed.java` y `ClientTimed.java` .

Descripción del entorno

	Computadora A (Servidor)	Computadora B (Cliente)
CPU	TODO: completar	TODO: completar
RAM	TODO: completar	TODO: completar
SO	TODO: completar	TODO: completar
Java	TODO: completar	TODO: completar
Red	TODO: misma LAN / redes distintas	TODO: misma LAN / redes distintas

Cómo ejecutar los experimentos

En la Computadora A (servidor):

```
javac ServerTimed.java
java ServerTimed 5555 srv_fisico.csv
```

En la Computadora B (cliente):

```
javac ClientTimed.java
for n in 10 100 1000 10000 100000 1000000; do
    java ClientTimed <IP_COMPUTADORA_A> 5555 $n cli_fisico.csv
done
```

Resultados

TODO: Completar con los datos reales de la corrida.

Generar gráficos con: `python3 plot_times_tp2.py cli_fisico.csv srv_fisico.csv pingpong_fisico.csv`

Análisis (Punto 2c)

Los resultados esperables son los mismos del análisis del TP1, con estas diferencias al ejecutar en red física:

- **OutputStream.write()** : En Java, `OutputStream.write(byte[])` bloquea hasta escribir todo el arreglo completo (a diferencia de C donde `write()` puede retornar escrituras parciales). Para tamaños chicos (10-1000 bytes), el tiempo está dominado por el overhead de la llamada al sistema subyacente, **no es proporcional** al tamaño. Para tamaños grandes (100K-1M bytes), el tiempo crece significativamente porque el buffer de envío del socket del kernel se llena y la JVM debe bloquearse esperando que TCP envíe y reciba ACKs del otro extremo para liberar espacio.
- **InputStream.read()** : El tiempo por llamada individual (buffer fijo de 255 bytes) se mantiene **aproximadamente constante**, ya que lo que cambia es la cantidad de llamadas, no el costo de cada una. En red física, el piso de tiempo por `read()` será más alto que en loopback por la latencia de red.
- **Proporcionalidad**: No es válido asumir que 1000 bytes tarda 10× más que 100 bytes. La relación no es lineal por el costo fijo de la syscall subyacente y el comportamiento de los buffers TCP.

Diferencia clave respecto a C: En Java, `out.write(data)` donde `data` es un `byte[]` siempre escribe la totalidad del arreglo en una sola llamada lógica (internamente la JVM puede fragmentarla, pero desde el punto de vista del programador es una única invocación). En C, `write(fd, buf, n)` puede retornar un valor menor a `n` (escritura parcial) y se necesita un bucle de reintentos. Esto hace que el código Java sea más simple, pero la medición de tiempo en el lado del escritor captura exactamente lo mismo: el tiempo que tarda el sistema operativo en aceptar los bytes en el buffer de envío del socket.

Ejercicio 3 — Experimento Ping-Pong (RTT / 2)

Concepto

El experimento “ping-pong” consiste en:

1. El cliente envía N bytes al servidor.
2. El servidor recibe los N bytes completos y los **reenvía de vuelta** al cliente.
3. El cliente mide el **tiempo total de ida y vuelta (RTT)**.
4. Se estima el tiempo en una dirección como **RTT / 2**.

Código desarrollado

Se crearon dos programas en Java:

ServerPingPong.java

```
// Protocolo:  
// 1. Lee 4 bytes (int) con DataInputStream.readInt() — total de bytes a recibir.  
// 2. Recibe los N bytes completos en un byte[] dinámico.  
// 3. Reenvía los N bytes completos de vuelta con out.write(buffer, 0, received).
```

El servidor crea un arreglo `byte[]` del tamaño exacto que le anuncia el cliente con `readInt()`, lee todos los bytes en un bucle (porque `InputStream.read()` puede retornar menos bytes de los pedidos), y luego los reescribe completos con una sola llamada a `out.write()` seguida de `out.flush()`.

¿Por qué un arreglo del tamaño total? A diferencia del `ServerTimed` que lee en chunks de 255 bytes (para medir cada `read()` individual), acá necesitamos acumular todos los datos recibidos para poder reenviarlos. El tamaño puede ser hasta 1MB.

Diferencia con C: En C se usa `malloc()` para asignar memoria dinámica y hay que liberar con `free()`. En Java, el garbage collector se encarga automáticamente de la memoria cuando el arreglo deja de usarse. Además, en Java `out.write(buffer, 0, received)` escribe el arreglo completo sin necesidad de un bucle de escritura parcial como en C.

ClientPingPong.java

```
// Medición:  
// t0 = System.nanoTime()  
// out.write(data) + out.flush() -- envío completo  
// in.read() en bucle -- recepción del eco completo  
// t1 = System.nanoTime()  
// RTT = t1 - t0  
// T_unidireccional ≈ RTT / 2
```

El cliente mide el RTT **desde antes del `out.write()` hasta después de recibir el último byte de vuelta**. Esto captura el tiempo total de ida (write + transmisión + recepción en servidor + procesamiento) y vuelta (reenvío + transmisión + recepción en cliente).

Sobre `System.nanoTime()` : Es un reloj monotónico de alta precisión (equivalente a `clock_gettime(CLOCK_MONOTONIC)` en C). No se ve afectado por cambios en la hora del sistema (NTP, usuario, etc.), lo que lo hace ideal para medir intervalos de tiempo. La resolución típica es de nanosegundos, aunque la precisión real depende del hardware.

Cómo ejecutarlo

En la computadora servidor:

```
javac ServerPingPong.java
java ServerPingPong 5556
```

En la computadora cliente:

```
javac ClientPingPong.java
for n in 10 100 1000 10000 100000 1000000; do
    java ClientPingPong <IP_SERVIDOR> 5556 $n pingpong.csv
done
```

Comparación Punto 2 vs Punto 3

La diferencia clave entre los dos métodos es:

Aspecto	Punto 2 (<code>write()</code> directo)	Punto 3 (RTT/2)
Qué mide	Solo el tiempo de <code>OutputStream.write()</code> local	Tiempo de ida + vuelta completo
Incluye red	Parcialmente (si el buffer se llena)	Siempre
Supuesto	Ninguno	Simetría del canal (ida = vuelta)
Valor esperado	$RTT/2 \geq \text{write}()$ siempre	Mayor que <code>write()</code> solo

RTT/2 siempre será $\geq$ al tiempo de `write()` del Punto 2, porque RTT/2 incluye:

- El tiempo de transmisión por la red (que `write()` no mide si cabe en el buffer).
- El tiempo de procesamiento del servidor.
- El tiempo de transmisión de vuelta.

La estimación $RTT/2$ **asume simetría del canal**: que el tiempo de ida es igual al de vuelta. En redes reales (especialmente WiFi o conexiones asimétricas como ADSL), esta suposición puede no ser exacta.

Resultados

TODO: Completar con los datos reales. Generar el gráfico comparativo con:

```
python3 plot_times_tp2.py cli_fisico.csv srv_fisico.csv pingpong.csv
```

Ejercicio 4 — Scripts de despliegue con Vagrant

El problema del asincronismo

Cuando se automatizan los experimentos con Vagrant, ambas VMs se aprovisionan secuencialmente (o en paralelo). El problema es: **¿qué pasa si el proceso cliente intenta conectarse antes de que el servidor haya hecho `bind()` + `listen()` ?**

En Java, el constructor `new Socket(ip, port)` (que internamente hace el `connect()`) lanza una excepción `java.net.ConnectException: Connection refused` porque el kernel del servidor no tiene un socket pasivo en ese puerto y responde con un paquete `RST` al `SYN` del cliente.

En C, el equivalente es que `connect()` retorna -1 con `errno = ECONNREFUSED`.

Solución implementada: doble mecanismo de sincronización

Se resolvió con **dos capas de verificación** en `run_client.sh`:

Capa 1: Señalización por archivo compartido

```
# El servidor crea este archivo cuando está listo:
echo "ready" > /vagrant/.server_ready

# El cliente espera a que exista:
while [ ! -f /vagrant/.server_ready ]; do
```

```
sleep 2
done
```

Funciona porque `/vagrant` es una carpeta compartida entre ambas VMs (y el host). Cuando el servidor escribe el archivo, el cliente lo ve inmediatamente a través del filesystem compartido de VirtualBox.

Capa 2: Verificación de puerto TCP con reintentos

```
# Después de que el archivo exista, verificar que el puerto esté realmente escuchando:
while ! (echo > /dev/tcp/$host/$port) 2>/dev/null; do
    sleep 2
done
```

Esta segunda capa es necesaria porque el archivo podría crearse *antes* de que el proceso `ServerTimed` realmente haya ejecutado `new ServerSocket(port)` (que internamente hace `bind()` + `listen()`). La verificación con `/dev/tcp` intenta abrir una conexión TCP real al puerto y solo continúa cuando tiene éxito.

Escenario 4a: Dos máquinas virtuales (VM a VM)

red privada		
vm_server	192.168.56.10/11	vm_client
ServerTimed:5555		ClientTimed
ServerPP:5556		ClientPingPong
/vagrant (compartida)		

Vagrantfile.punto4a levanta dos VMs con IPs fijas en una `private_network` :

- `vm_server` (192.168.56.10): compila (`javac *.java`) y lanza los servidores Java.
- `vm_client` (192.168.56.11): compila los clientes Java, espera al servidor, y ejecuta todos los experimentos automáticamente.

Los resultados quedan en `/vagrant/results/` (= la carpeta local `results/`).

Cómo ejecutarlo:

```
cp Vagrantfile.punto4a Vagrantfile
vagrant up
# Los experimentos se ejecutan automáticamente durante el provisioning.
# Los resultados quedan en results/
ls results/
```

Contenido del script `run_server.sh` (adaptado a Java):

```
#!/bin/bash
cd /vagrant

# Compilar todos los archivos Java
javac *.java

# Lanzar servidores en background
java ServerTimed 5555 results/srv_results.csv &
java ServerPingPong 5556 &

# Señalizar que los servidores están listos
sleep 2 # dar tiempo a que los sockets se abran
echo "ready" > /vagrant/.server_ready
echo "[run_server] servidores Java lanzados, señal enviada."
```

Contenido del script `run_client.sh` (adaptado a Java):

```
#!/bin/bash
HOST="192.168.56.10"
cd /vagrant
mkdir -p results

# Compilar
javac *.java

# Capa 1: esperar archivo de señal
while [ ! -f /vagrant/.server_ready ]; do
    echo "[run_client] esperando señal del servidor..."
    sleep 2
done
```

```
# Capa 2: verificar que el puerto esté abierto
while ! (echo > /dev/tcp/$HOST/5555) 2>/dev/null; do
    echo "[run_client] esperando que el puerto 5555 esté escuchando..."
    sleep 2
done

# Ejecutar experimentos
for n in 10 100 1000 10000 100000 1000000; do
    java ClientTimed $HOST 5555 $n results/cli_results.csv
    java ClientPingPong $HOST 5556 $n results/pingpong_results.csv
done

echo "[run_client] experimentos completados."
```

Escenario 4b: Una VM + el host

		forwarded_port	
VM (servidor)	5555	5555	HOST
	5556	5556	(cliente)
ServerTimed:5555			ClientTimed
ServerPP:5556			ClientPingPong

Vagrantfile.punto4b levanta una sola VM con servidores Java en los puertos 5555 y 5556, redirigidos al host mediante `forwarded_port` . El cliente se ejecuta desde el host contra `127.0.0.1` .

Cómo ejecutarlo:

```
cp Vagrantfile.punto4b Vagrantfile
vagrant up
# Los servidores Java ya están corriendo en la VM.
# Desde el host (necesita Java / JDK instalado):
bash run_client_host.sh
```

Ventaja de Java en este escenario: El mismo bytecode Java compilado en la VM funciona en el host sin recompilación (portabilidad “write once, run anywhere”), siempre que el host tenga un JDK/JRE compatible. En C, habría que recompilar para cada sistema operativo y arquitectura.

Consulta a la IA: mejor método para estimar tiempos de comunicación

Pregunta realizada a Claude (Anthropic):

“¿Cuál es el mejor método experimental para estimar el tiempo de comunicación entre dos computadoras diferentes?”

Respuesta resumida de la IA:

El método más robusto es el de **ping-pong (RTT/2)** con las siguientes mejoras:

1. **Múltiples iteraciones:** Repetir cada medición al menos 100 veces y calcular la mediana (no el promedio, para evitar outliers por jitter de red o scheduling del SO).
2. **Descarte de warm-up:** Las primeras mediciones suelen tener latencia elevada por resolución ARP, establecimiento de conexión TCP, y cachés frías. Descartarlas. **En Java esto es especialmente importante** por el JIT compiler (Just-In-Time): las primeras ejecuciones son interpretadas y más lentas; después de varias iteraciones el JIT compila el código a nativo y el rendimiento mejora significativamente.
3. **Sincronización de relojes:** Si se quiere medir unidireccionalmente, usar NTP o PTP para sincronizar los relojes de ambas máquinas. Sin embargo, NTP tiene una precisión típica de $\pm 1\text{-}10\text{ms}$, lo cual puede ser mayor que la propia latencia a medir en una LAN.
4. **Control del entorno:** Deshabilitar power saving (CPU frequency scaling), fijar la frecuencia del CPU, y minimizar tráfico de red concurrente.

Comentario propio / Crítica de la respuesta:

La respuesta de la IA es mayormente correcta, pero tiene un problema potencial: **la suposición de simetría de RTT/2 no siempre es válida**. En redes WiFi, la latencia de subida puede ser significativamente diferente a la de bajada. Además, la IA no menciona que en entornos virtualizados (como nuestro caso con Vagrant/VirtualBox), la virtualización del adaptador de red agrega una capa adicional de variabilidad que no existe en hardware real, lo que puede hacer que los resultados sean menos reproducibles.

Otro punto que la IA no enfatiza suficientemente es el **efecto del tamaño del mensaje en la simetría**: para mensajes muy grandes (1MB), el tiempo de transmisión domina y la simetría se mantiene mejor; para mensajes muy chicos (10 bytes), el overhead del protocolo y el jitter son proporcionalmente mayores, haciendo la estimación RTT/2 menos precisa.

Nota adicional sobre Java: Al usar Java en lugar de C, hay una capa adicional a considerar: la JVM. Aunque los sockets Java usan las mismas llamadas del sistema

operativo por debajo (`socket()` , `bind()` , `connect()` , `read()` , `write()`), la JVM agrega un overhead mínimo por la indirección. Sin embargo, para mediciones de red (donde la latencia de red domina), este overhead es despreciable.

Ejercicio 5 — Impacto del orden de ejecución y el desfase temporal

Pregunta del enunciado

¿Los resultados de tiempo se ven afectados por el orden de ejecución y las diferencias de tiempo iniciales entre los programas? ¿Qué sucede si hay una diferencia de 10 segundos en el inicio?

Experimento diseñado

Se creó `ClientSleepTest.java` con 4 modos de operación, cada uno insertando un `Thread.sleep(10000)` (10 segundos) en un punto diferente del ciclo de vida de la conexión:

Modo	Dónde se inserta el <code>Thread.sleep()</code>	Efecto esperado
0	Sin sleep (baseline)	Referencia para comparar
1	ANTES de <code>new Socket()</code> (= antes de <code>connect</code>)	El servidor ya hizo <code>new ServerSocket()</code> . El cliente simplemente tarda 10s más en conectar. No afecta los tiempos de <code>write/read</code> una vez conectado.
2	DESPUÉS de <code>new Socket()</code> , ANTES de <code>out.write()</code>	La conexión TCP ya está establecida (3-way handshake completado). El servidor está bloqueado en <code>in.read()</code> esperando datos. No afecta los tiempos de transferencia.
3	DESPUÉS de <code>out.write()</code> , ANTES de <code>in.read()</code>	El cliente envió todos los datos. El servidor recibe, procesa y reenvía los datos. Los datos de vuelta llegan al buffer de recepción del kernel del cliente y quedan ahí esperando durante 10 segundos. Cuando el <code>in.read()</code> finalmente se ejecuta, los datos ya están en el buffer local el <code>read()</code> será más rápido de lo normal (casi instantáneo, sin esperar la red).

Explicación técnica

¿Por qué el sleep antes de connect/write no afecta las mediciones?

Porque la medición de tiempos se hace **alrededor de cada operación individual**, no del programa completo:

```
// El sleep ocurre FUERA de la zona medida:
Thread.sleep(sleepSecs * 1000L);      // no se mide esto
long tw0 = System.nanoTime();         // inicio de medición
out.write(data);
out.flush();
long tw1 = System.nanoTime();         // fin de medición
```

El `Thread.sleep()` solo retrasa el momento en que se ejecuta la operación, pero no cambia la velocidad de la red ni el tamaño de los buffers.

¿Qué pasa con el sleep DESPUÉS del write (Modo 3)?

Este es el caso más interesante. Lo que sucede durante esos 10 segundos:

1. El cliente hizo `out.write(data)` de N bytes los datos viajan al servidor por TCP.
2. El servidor recibe los N bytes completos y hace `out.write(buffer)` de vuelta.
3. Los datos de vuelta viajan del servidor al cliente **llegan al buffer de recepción del kernel del cliente** (`SO_RCVBUF`).
4. El cliente está durmiendo (`Thread.sleep(10000)`) nadie llama a `in.read()` .
5. Los datos se acumulan en el buffer del kernel. Si $N < \text{tamaño del buffer}$ (típicamente 128K-256K), todo cabe sin problema.
6. Cuando el `Thread.sleep()` termina y el cliente llama a `in.read()` , los datos **ya están en memoria local** la lectura es casi instantánea (solo copia de kernel a espacio de usuario, sin esperar la red).

Resultado: El `in.read()` del Modo 3 será significativamente **más rápido** que el baseline, porque no incluye el tiempo de red (ya llegaron los datos).

Nota Java-específica: `Thread.sleep()` es equivalente funcionalmente a `sleep()` de C (ambos suspenden el hilo/proceso). La diferencia es que en Java se opera con milisegundos (`Thread.sleep(10000)` = 10 segundos) mientras que en C `sleep()` opera con segundos (`sleep(10)` = 10 segundos).

¿Y si el buffer se llena? (caso de 1MB)

Si los N bytes de vuelta son mayores que el buffer de recepción, TCP usa su mecanismo de **control de flujo por ventana (TCP Window Advertisement)**:

- El buffer del receptor se llena.
- El receptor anuncia una ventana de 0 bytes.
- El emisor (servidor) se bloquea en su `out.write()` esperando que el receptor libere espacio.
- Cuando el cliente despierta y hace `in.read()` , libera espacio en el buffer el servidor puede seguir enviando.

En este caso, el `Thread.sleep(10000)` del Modo 3 **sí afecta** el rendimiento global porque fuerza al servidor a pausar su transmisión.

Este comportamiento es **idéntico en Java y en C** porque el control de flujo TCP ocurre a nivel del kernel del sistema operativo, no del lenguaje.

Cómo ejecutar el experimento

```
# En una terminal: servidor pingpong
java ServerPingPong 5556

# En otra terminal: experimento completo
# Para cada modo (0-3) y cada tamaño:
for modo in 0 1 2 3; do
  for n in 1000 100000 1000000; do
    java ClientSleepTest 127.0.0.1 5556 $n $modo 10 results/sleep_experiment.csv
  done
done
```

Conclusión del Punto 5

Situación	¿Afecta los tiempos de transferencia?
Servidor arranca 10s antes que el cliente	No. El servidor espera en <code>serverSocket.accept()</code> hasta que el cliente conecte.
Cliente arranca 10s antes que el servidor	Sí <code>new Socket(ip, port)</code> lanza <code>ConnectException: Connection refused</code> . Por eso se necesita la solución de asincronismo del Punto 4.

Sleep antes de connect (modo 1)	No afecta write/read.
Sleep antes de write (modo 2)	No afecta write/read.
Sleep después de write, antes de read (modo 3)	Sí afecta read: lo hace artificialmente más rápido porque los datos ya están en el buffer local. Para datos > tamaño del buffer, también afecta al emisor (control de flujo TCP).

Justificación de la elección de Java sobre C

Para este TP se eligió Java sobre C por las siguientes razones técnicas:

Aspecto	Java	C
Escritura en socket	<code>out.write(byte[])</code> escribe todo el arreglo en una llamada. Sin escrituras parciales a nivel de API.	<code>write()</code> puede retornar escrituras parciales. Necesita bucle de reintentos.
Lectura en socket	<code>in.read(buf, off, len)</code> puede retornar menos bytes. mismo bucle que C.	<code>read()</code> puede retornar menos bytes. mismo bucle.
Memoria	Garbage collector automático. <code>byte[] buf = new byte[n]</code> y listo.	<code>malloc()</code> + <code>free()</code> manuales. Riesgo de memory leaks.
Manejo de errores	Excepciones (try/catch). El programa no se cuelga silenciosamente.	Valores de retorno + <code>errno</code> . Riesgo de SIGSEGV.
Medición de tiempo	<code>System.nanoTime()</code> — monotónico, nanosegundos.	<code>clock_gettime(CLOCK_MONOTONIC)</code> — mismo concepto.
Portabilidad	Mismo bytecode corre en Linux, Windows y macOS sin recompilar.	Hay que recompilar para cada plataforma.

Scripting	<code>javac *.java && java Clase args</code> — una línea por script.	<code>gcc -O2 -o prog prog.c && ./prog args</code> — similar.
Red subyacente	Los sockets Java usan las mismas syscalls del SO (<code>socket()</code> , <code>connect()</code> , <code>read()</code> , <code>write()</code>).	Llamadas directas al kernel.

Conclusión: Para experimentos de red donde lo que importa es medir la latencia de la comunicación TCP (no el overhead de CPU o memoria), Java y C producen resultados equivalentes porque ambos usan las mismas llamadas del sistema operativo por debajo. Java ofrece un código más limpio, seguro y fácil de entender/defender en una entrega.

Archivos entregados

Código fuente (Java)

Archivo	Descripción	Punto
<code>ServerTimed.java</code>	Servidor que mide tiempos de cada <code>read()</code> (buffer 255 bytes)	2
<code>ClientTimed.java</code>	Cliente que mide tiempo de <code>write()</code> completo	2
<code>ServerPingPong.java</code>	Servidor eco (recibe N bytes, reenvía N bytes)	3
<code>ClientPingPong.java</code>	Cliente que mide RTT (write + read ida y vuelta)	3
<code>ClientSleepTest.java</code>	Cliente con <code>Thread.sleep()</code> configurable en 4 modos	5

Scripts de automatización

Archivo	Descripción	Punto
<code>run_server.sh</code>	Compila y lanza ambos servidores Java en background	4
<code>run_client.sh</code>	Compila clientes, espera activa + todos los experimentos	4

run_client_host.sh	Cliente para escenario VM Host	4b
run_sleep_experiment.sh	Automatiza los 4 modos de sleep	5

Configuración Vagrant

Archivo	Descripción	Punto
Vagrantfile.punto1	Ubuntu 24.04 LTS (Noble) — verificación de versión + JDK	1
Vagrantfile.punto4a	2 VMs (VM a VM) con red privada	4a
Vagrantfile.punto4b	1 VM (servidor) + host (cliente) con forwarded_port	4b

Graficación

Archivo	Descripción
plot_times_tp2.py	Genera los 4 gráficos del TP2 a partir de los CSVs